



## Lecture 40

# Wrapping Things Up

CS61B, Spring 2026 @ UC Berkeley

# 61B in 12 Minutes or Less

---

Lecture 40, CS61B, Spring 2026

## 61B in 12 Minutes or Less

Your Future

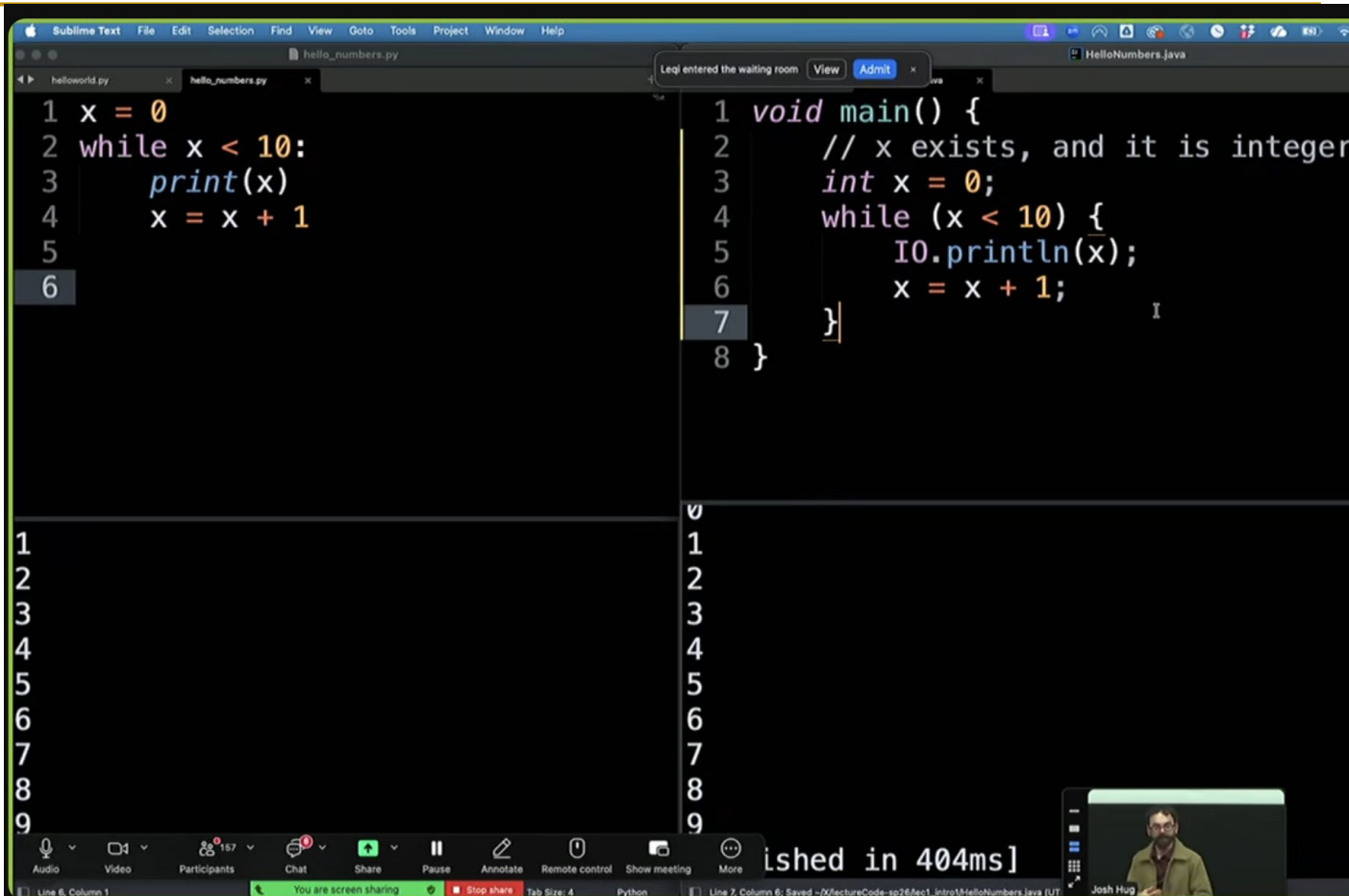
Course Reflections

AMA

So Long!

# Where We Started

Just 14 weeks ago:



```
1 x = 0
2 while x < 10:
3     print(x)
4     x = x + 1
5
6
```

```
1 void main() {
2     // x exists, and it is integer
3     int x = 0;
4     while (x < 10) {
5         IO.println(x);
6         x = x + 1;
7     }
8 }
```

ished in 404ms]

## What We've Learned about Programming Languages

---

- Object oriented programming:
    - Interface Inheritance. 
    - Implementation inheritance.
  - Generic Programming, e.g. ArrayList<Integer>, etc.
  - The model of memory as boxes containing bits. 
  - Bit representation of positive integers (hashing lecture).
  - Java.
  - Some standard programming idioms/patterns:
    - Objects as function containers (e.g. Comparators).
    - Default method specification in interfaces.
    - Iterators.
- Example: Programmer only needs to know List API, doesn't have to know that ArrayList secretly does array resizing.**
- Example: Array is a sequence of boxes. An array variable is a box containing address of sequences of boxes.**

Important data structure interfaces:

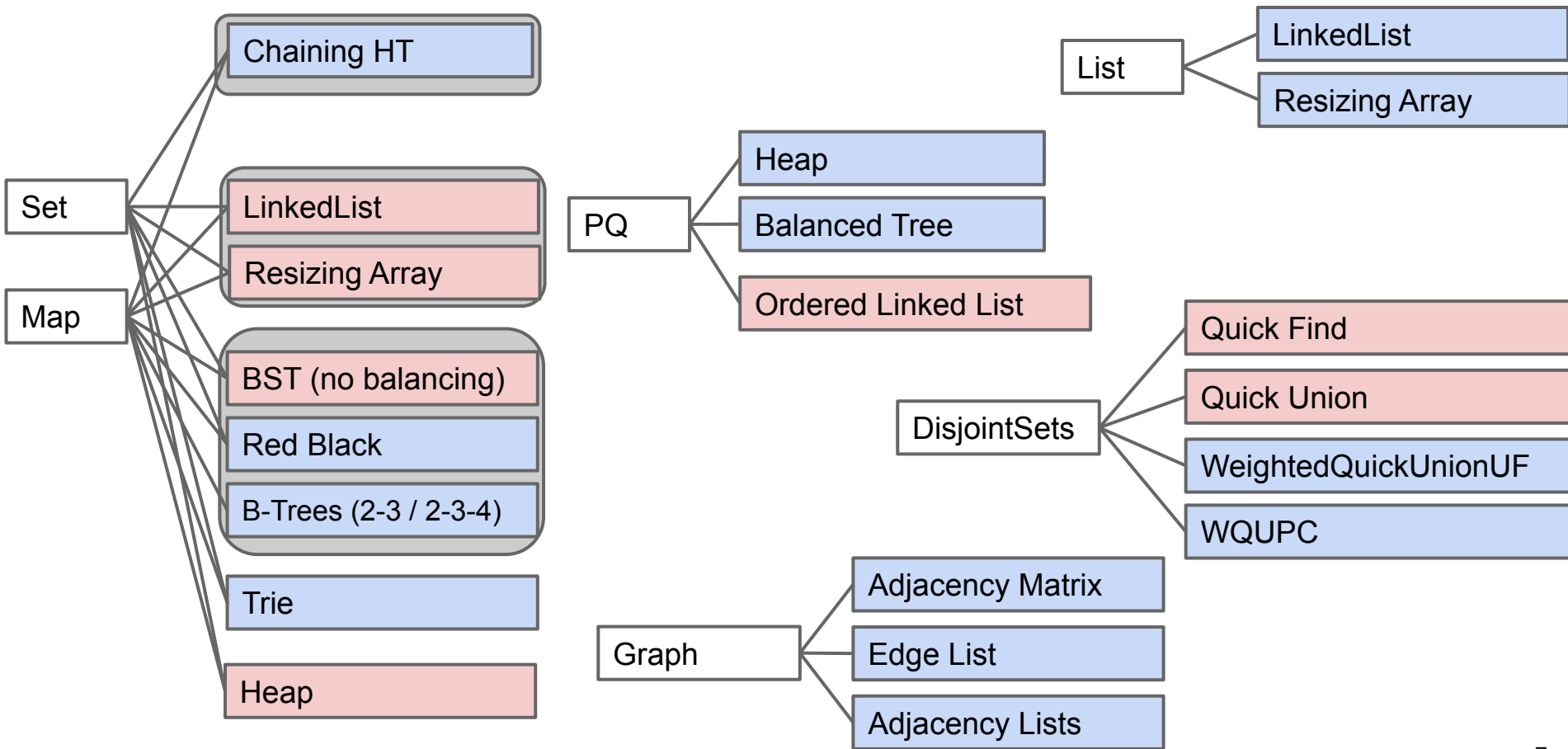
- `java.util.List`
- `java.util.Set`
- `java.util.Map`
- Our own Collections (e.g. `Map61B`, `Deque61B`)

Concrete implementations of these abstract implementations:

- Examples: `ArrayDeque61B` implements `Deque61B`.

- Asymptotic analysis.
- $O(\cdot)$ ,  $\Omega(\cdot)$ , and  $\Theta(\cdot)$ .
- Worst case vs. average case vs. best case.
  - Exemplar of usefulness of average case: Quicksort
- Determining the runtime of code through inspection (often requires deep thought).
- Multiplicative resizing is much better than additive resizing.
  - Multiplicative resizing results in  $\Theta(1)$  add runtime for ArrayLists.
  - Additive resizing results in  $\Theta(N)$  add runtime for ArrayLists.

Some Examples of Implementations for ADTs



### Array-Based Data Structures:

- ArrayLists and ArrayDeque
- HashSets, HashMaps, MyHashMap: Arrays of 'buckets'
- ArrayHeap (tree represented as an array)

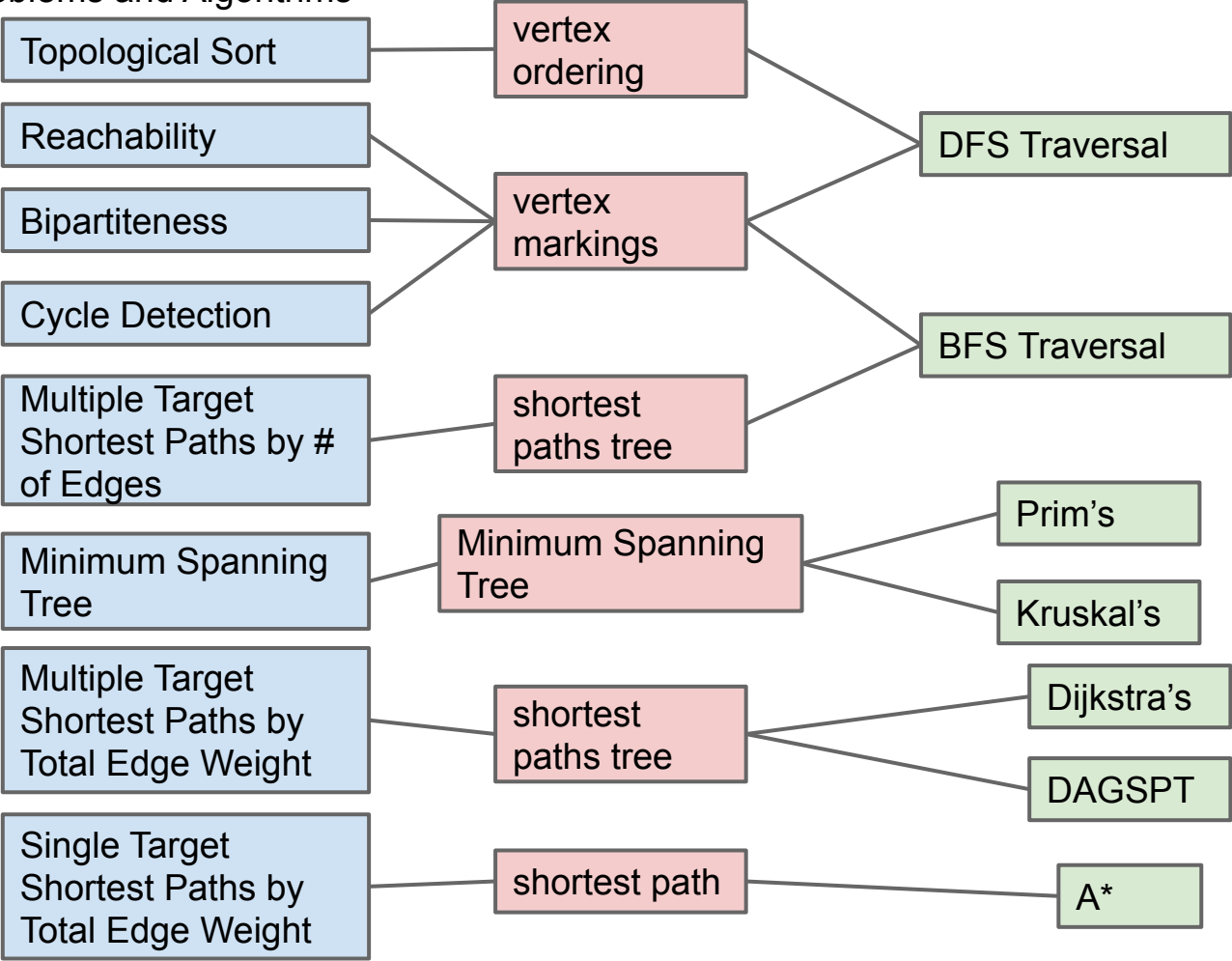
### Linked Data Structures

- Linked Lists
  - LinkedList, IntList, LinkedListDeque, SLList, DLList
- Trees: Hierarchical generalization of a linked list. Aim for bushiness.
  - TreeSet, TreeMap, BSTMap, Tries (trie links often stored as arrays)
- Graphs: Generalization of a tree (including many algorithms).

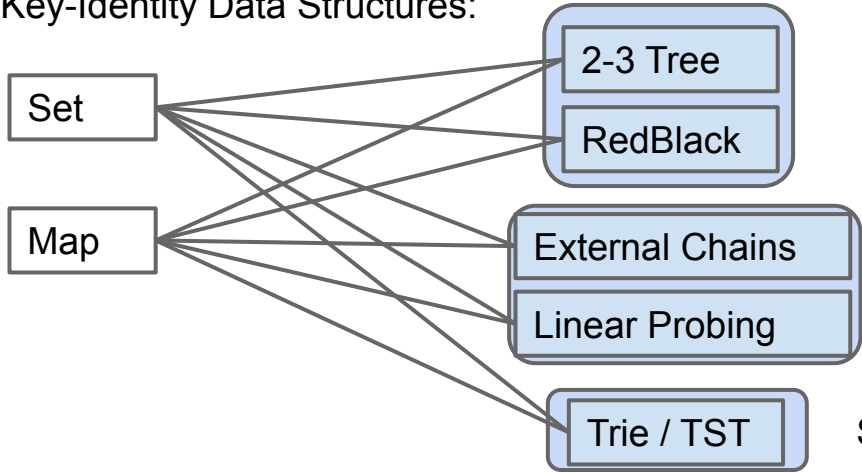
Tradeoffs of array-based vs. linked data structures.



Tractable Graph Problems and Algorithms



Search-By-Key-Identity Data Structures:

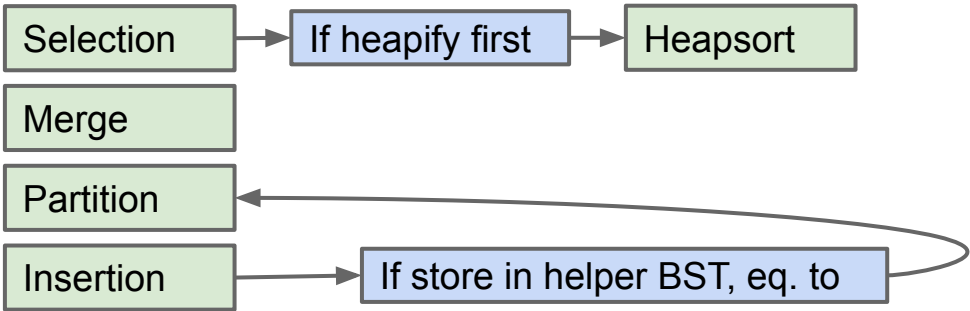


Searches using compareTo()  
Analogous to **Comparison-Based**

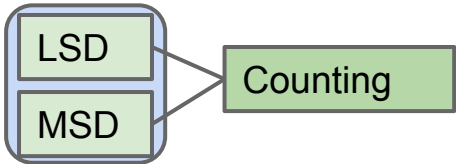
Searches using hashCode() and equals()  
Roughly Analogous to **Integer Sorting**

Searches by digit. Analogous to **Radix Sorting**

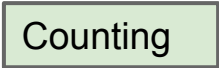
**Comparison Based** Sorting Algorithms:  $\Omega(N \log N)$  worst case



**Radix Sorting** Algorithms:  $\Theta(NW)$  worst case:  
(require a sorting subroutine)



**Small-Alphabet (Integer) Sorting Algorithms:**  $\Theta(N)$  worst case



- Java syntax and idioms.
- Unit testing (and its more extreme form: Test-driven development).
- Asking the internet and LLMs for help.
- Debugging:
  - Identify the simplest case affected by the bug.
  - Hunt it down, giving it no place to hide.
  - With the right methodology, can find bugs even when finding bug through manual code inspection is impossible.
- Tactical vs. Strategic Programming.
  - Managing complexity. Hiding complexity. Deep interfaces.
- Real tools: IntelliJ, git, Truth, command line.
- Data structure selection (and API Design)
  - Drive the performance and implementation of your entire program.
- LLMs: This is how you'll write code in the future, but not in 61B.

# Your Future

---

Lecture 40, CS61B, Spring 2026

61B in 12 Minutes or Less

**Your Future**

Course Reflections

AMA

So Long!

Two interesting questions:

- What can you do?
- What should you do?

## Some Josh Hug History

[illegible]

My strong belief: With sufficient high quality deliberate practice, you can all get **very good** at any topic you study here at Berkeley.

But Berkeley classes move at a fixed pace! So if you start behind, it is extremely hard to catch up.

Your grade in this class tells you how prepared you are for your downstream programming-oriented CS class.

- Example: you had a B or lower in this class, you'll need a LOT of prep to feel comfortable in 61C, 16X, etc.

## Some Project 0 Demos

---

Ana Abell: <https://www.youtube.com/watch?v=B7u7FrQ1yDY>

Audrey Kramer: [https://www.youtube.com/watch?v=cg\\_a50V6410](https://www.youtube.com/watch?v=cg_a50V6410)

Zinan Wang: <https://www.youtube.com/watch?v=yyoL0LJpEPQ>

Janet Padilla: <https://www.youtube.com/shorts/-DTUTOnkHCw>

Miles Keiser: <https://www.youtube.com/watch?v=E2d92-M2BnM>

Alice Kuznetsov: <https://www.youtube.com/watch?v=N5BM0bXixHY>



People are going to want to hire you.

- Big companies.
- Startups.
- Government agencies.
- Non-profits.

You get to choose where to go.

Quite a lot of you will likely end up working at some sort of technology company at some point in your life.

There's nothing (IMO) wrong with working at profit driven tech companies.

- Please do realize that even as a rank and file employee, you have the power to effect change, particularly if you are paid in stock (because then you are a partial owner). Some Meta examples:
  - Sectarian violence in Myanmar driven by Facebook posts was a huge problem due to a lack of local language speakers. Nobody at Meta stepped up to fix this for years.
  - Frances Haugen (former Meta data scientist) leaked documents showing shady behavior by the company.

And of course realize that your skills can be used in many ways!

My request: Use your superpowers in a way that is a net positive for the lives of your fellow humans.

- I'm not saying that working for big companies is evil or anything, or that you have a moral obligation to work for relatively low wages for the public good (though that'd be great for the world, of course).
- .... but keep in mind that your work will profoundly affect the world.

(Wanna talk about this stuff more? Take CS195 next Spring!)

Don't waste it. There's an infinite number of cool things to do.

### Another fun idea: Dutch 1

- M/W: 12 - 2 PM
- F: 12 - 1 PM
- In person and/or Zoom.
- Class #: 21909

# DUTCH 1

## *Elementary Dutch*



Image: ievanderhoeven

Interested in learning an easy language in between English and German? Consider taking Dutch 1! In this beginner's course, you will familiarize yourself with the basics of Dutch: its sounds and spelling, its grammatical structure, and its vocabulary. The class focuses on oral communication with an emphasis on vocabulary: Learning words and learning how to use these words. By reading texts and dialogues (and listening to the audio version), you will build your vocabulary. In class you will get the opportunity to practice your newly learned words and phrases. By the end of the semester, you will be able to express yourself in speaking and writing about a variety of topics, including introducing yourself, talking about your day, getting around, food and drinks, your house, and more.

M-W 12-2pm + F 12-1pm | 5 units

This course will be offered in person on campus and on Zoom.  
Students from other UCs can enroll via UC Online.

*For more information, please contact the instructor:*

Esmée van der Hoeven, [ievanderhoeven@berkeley.edu](mailto:ievanderhoeven@berkeley.edu)

*Or visit:* [dutch.berkeley.edu](http://dutch.berkeley.edu)

Fall  
2026

# Course Reflections

---

Lecture 40, CS61B, Spring 2026

61B in 12 Minutes or Less  
Your Future

## Course Reflections

AMA

So Long!

### CS61B Version 4.5 (Spring 2026)

- New project 0: Particles.
  - And [Secret Project X](#) (Awakening of Azathoth).
- Elimination of the swarm of small assignments added in Fall 2025.
- Computer based Midterm 1 with retakes.
- Personally wrote many discussion sections and taught a section.
- Much bigger focus on coming to things in person.
  - Keyword based matching to form better discussion groups.
  - 8 discussion sections in Fall 2025.
  - 33 discussion sections in Spring 2026.
  - ~40% discussion and lecture attendance.
- New discussion format: Much more active, less mini-lecture.
- Added lateness/point flexibility on design projects (Project 4 and 5).
- Project 2: Added LLM design doc grading.
- Project 3: Added pathfinder.

Main changes I have in mind:

- Bring back more formal lab section.
  - Including graded quizzes?
- Slight expansion of CBTF programming exam.
- Add ability to track progress on course material to website.
  - Including more “choose your own adventure” options, e.g. optional exercise where you can implement a Weighted Quick Union.
- Build nicer visualizations and interactive demos for every part of course.
  - Similar to the BST tree rotation game.
- Tentatively: Move from IntelliJ to a cloud based IDE.
  - Give feedback to students while working (or right after).
  - Reduce plagiarism (still quite bad!)
- Tentatively: Even more open-ended programming challenges for students with especially strong background (secret projects Y, Z, etc)
- Very tentatively: Have a HW or project where LLM use is the point.

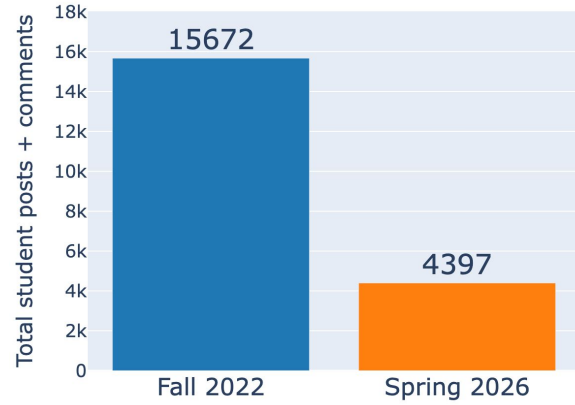


We read all of your feedback, especially on the end of course evaluations.

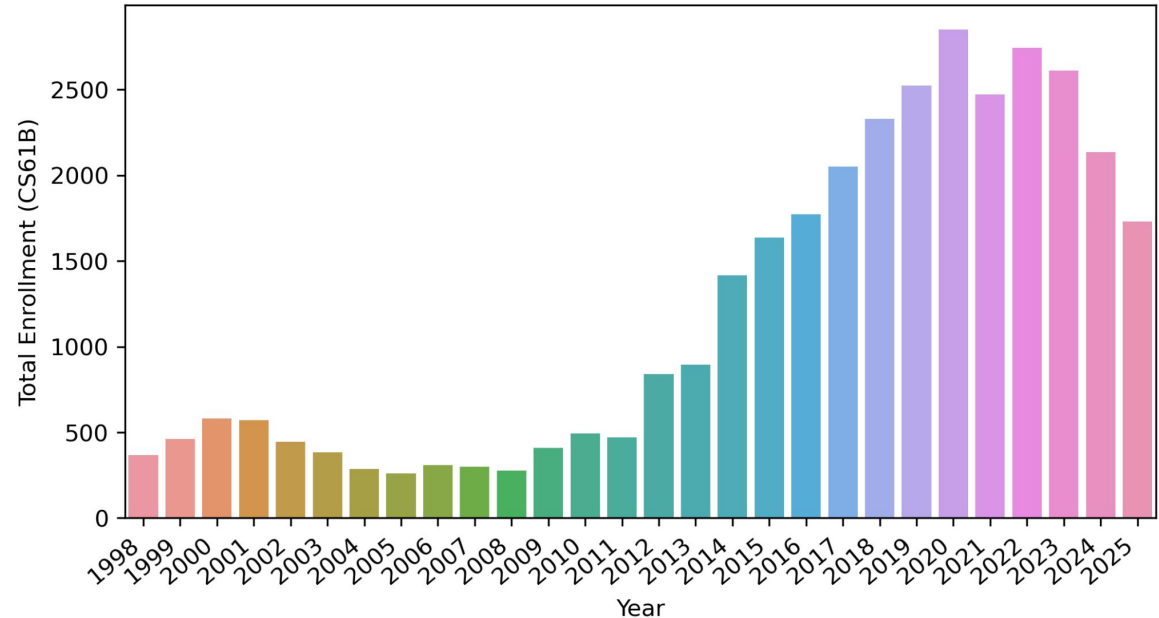
- There is an official form from the university, and of course the week 15 weekly survey.

## Interesting trends:

- Budget pressures forcing lower enrollment caps.
- Less interest in CS. No longer the obvious high paying career move.
- LLM use sharply reduced demand for staff time per capita.
- LLMs are dramatically changing how people work.

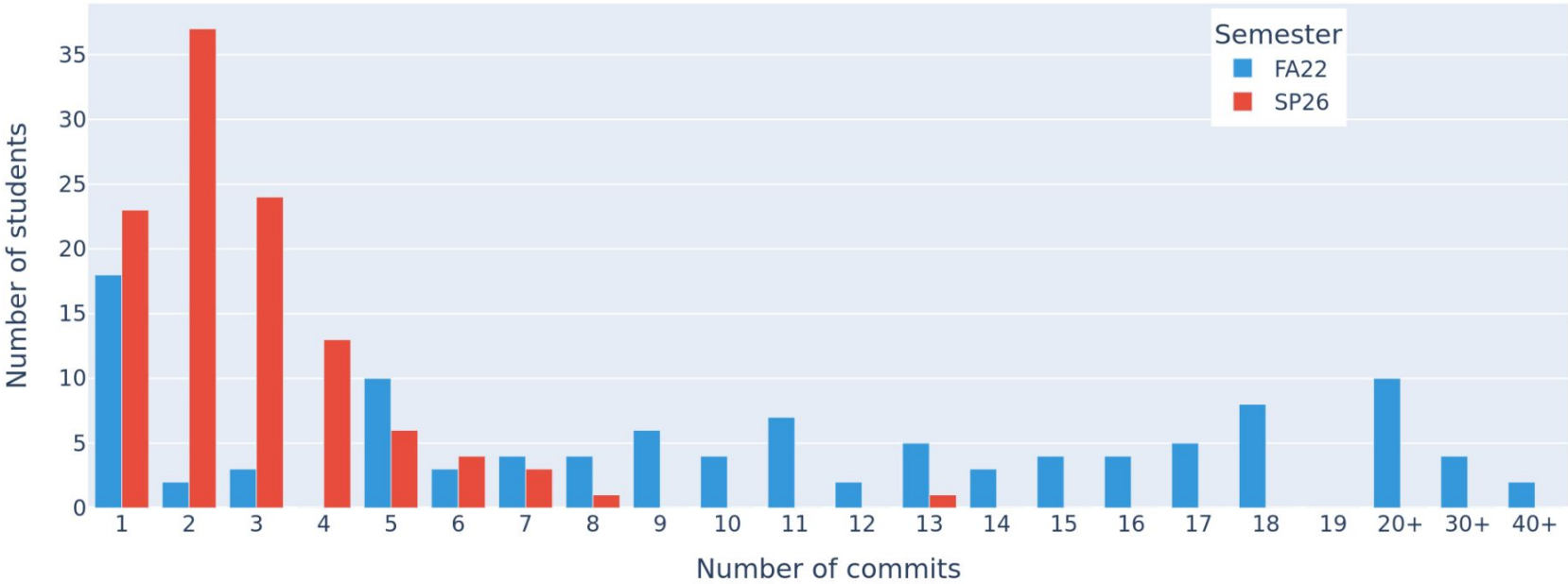


Ed use: Fa22 vs. Sp26  
same # students



# Fa22 vs. Sp26 Percolation

Distribution of Commits on Percolation Assignment: FA22 vs SP26



I am personally having more fun than I've ever had programming.

- And teaching is also incredibly exciting.

We'll see where this all goes.

# AMA

---

Lecture 40, CS61B, Spring 2026

61B in 12 Minutes or Less  
Your Future  
Course Reflections  
**AMA**  
So Long!

## Questions About Anything for Josh?

---

What sensitive info do you let Claude see?

- For student data, I give it anonymized data.
- For my own data, I let it see my credit card statements, why not?

What is the rationale for limiting submissions?

- So you don't hammer the autograder. Maybe Fa22 had no time limit at all.

Why not put tests on Github?

- Why not use github actions or similar? I could revisit. Gradescope already existed.

Pros and Cons of LLMs:

- Opus 4.7 I know it is quite good. I bet ChatGPT 5.5 is almost as good or better I dunno.
- Opus 4.7 still does very dumb things, for example today I asked “why is teh waiting list for the Hajj so long in Indonesia”, and its answer was “Saudi Arabia gives only 1 slot per 1000 muslims in a country and Indonesia has the most Muslims, (and more stuff that did make sense)”
- I love the Agentic tools, I use Claude Code, but my colleague Koushik suggests using his KISS Sorcar.
- Cursor is good too.

# So Long!

---

Lecture 40, CS61B, Spring 2026

61B in 12 Minutes or Less

Your Future

Course Reflections

AMA

**So Long!**

Teaching interns: Learn more, help others, get units.

- Watch out for an announcement on the Spring 2026 61B Ed.



**Head TAs:** Amy Wang, Daniel Wang, Stacey Lei

**TAs:** Aditya Hariharan, Andrew Schoenen, Benjamin Yi, Dawn Schumacher, Dennis Koh, Iris Zhou, Isabel Dong, Karen Meng, Lilah Durney, Marcus Koh, Michelle Koga, Natalia Grondin, Ricardo Sandoval, Ritika Joshi, Rushil Saraf, Ryan Yang, Sarveshan Saravanan, Tiffany Li, Yinqi Yi

**Tutors:** Amanda Zhang, Andrew Xu, Avik Gautam, Deven Mittal, Esha Potharaju, Isaac Tieu, Ishanth Hombaiah, Miller Liu, Samuel Hudson, Selina Fang

**Teaching Interns (TIs):** Adam Hacker, Adrian Chu, Alice Wu, Andrew Choy, Angelina Wang, Ariel Shen, Caitlin Tong, Calvin Cha, Chinmaya Jagadeesh, Daniel Hsiung, Daniel Lief, Gabriel Moreno, Ganesh Karayi, Harsh Ambardekar, Khai Ho, Maya Sriram, Nathan Tran, Nithya Appannagaari, Smriti Rangarajan, Tarun Shah, Tommy Duong, Xiang Meng

It is a terrifying and awesome responsibility to decide how you should spend hundreds of your precious hours here at Berkeley.

I hope I've done a good job. I know it wasn't perfect (and sadly never will be).

- (but I've got Ideas™)

... and thanks for all the kind words, feedback, and understanding when things are not as good as they could be.